Can House Price Prediction Be Enhanced Using Property Description Text? A Regression-Based Approach

Alexander Wood
a.g.wood@student.vu.nl

Pradakshina Sharma
p.sharma2@student.vu.nl

Natasha Kanth
n.kanth@student.vu.nl

Abstract

Recent improvements in text processing have significantly increased the potential for unstructured data in prediction models. However, most house price prediction models are based on structured numerical features such as the number of rooms and the size of the property. This research aims to examine the potential of using property descriptions in improving the performance of house price prediction models. In this research, three regression algorithms will be implemented and compared using a real estate dataset. The algorithms will be Ridge Regression, Random Forest, and XGBoost. The algorithms will be implemented using two different sets of the dataset. One will be based solely on structured features, while the other will be based on structured features and property descriptions. The property descriptions will be processed using TF-IDF vectorization. The main purpose of using two different sets of the data set is to examine the potential of using property descriptions to improve the performance of house price prediction models.

1 Introduction

Individual financial decisions and more general economic activity are heavily influenced by the real estate markets. A wide range of stakeholders, including buyers, sellers and investors, benefit from accurate house price predictions because they promote transparent markets and well-informed decision-making. As a result, machine learning and data science researchers are now actively working on predicting home prices [1] [2] [3] [4].

The majority of current house price prediction models only use structured numerical and categorical features, such as area measurements, the number of stories, the number of bedrooms and bathrooms, the availability of a garage, the year the property was built, and the type of property. These features do not take into account the qualitative information that is often present in property descriptions, although they capture significant physical aspects of a property [2].

Details about a property's condition, style, neighbourhood atmosphere, and unique selling points are often included in listing descriptions; this information can have a big impact on perceived value but is challenging to process. This gap offers a chance to investigate whether combining unstructured text data with structured features can improve prediction performance. Text integration into regression models is becoming more feasible due to the development in natural language processing, especially TF-IDF vectorization [10]. This study looks into whether adding text to a property description enhances the accuracy of house price predictions. The evaluation metrics R2, RMSE, and MAE were used to measure performance on the test set. The research question being "Does adding written house descriptions improve the predictive accuracy of ML models for house price prediction" with a sub question "Which model out of ridge regression, random forest, XGBoost shows the greatest performance improvement when listing descriptions are added to structured features?"

2 Literature Review

While the structured features capture significant physical aspects of a property, they fail to account for the descriptive and contextual information that is frequently present in property listings and that can have a considerable impact on perceived value. Understanding the disadvantages of using only structured features, researchers have begun to explore the effectiveness of using property listing descriptions in their predictive models.

[1] have shown the importance of using property description in automated real estate valuation models. They demonstrated the predictive capability of the text-based attributes derived from the real estate property listing description, showing that un-

structured textual data carries meaningful signals that structured attributes alone are unable to capture. [2] further extended this research by using real estate property listings in five Canadian cities. They experimented using three word embedding techniques, TF-IDF, Word2Vec, and BERT. They demonstrated the effectiveness of using both text-based features and structured features in the real estate predictive models by showing that the combination of both feature types performed better than using only one feature type. They also demonstrated the effectiveness of using deep learning models in real estate prediction by using the Word2Vec word embedding technique, with their customized deep neural network achieving the strongest overall results when both feature types were combined. [3] proposed a natural language processing-based predictive model for house price forecasting. They demonstrated the predictive capability of text-based attributes derived from the real estate property listing description, further confirming that listing descriptions contribute meaningful predictive information beyond what structured attributes alone can provide. Recently, [4] proposed a multi-modal deep learning approach for real estate prediction using images, text, and structured attributes, demonstrating that jointly processing multiple data modalities simultaneously can further strengthen predictive performance.

Though these studies cumulatively prove that using textual data in house price prediction is more efficient, these studies are mainly based on deep learning approaches and sophisticated vectorization techniques like BERT and Word2Vec, which, while powerful, tend to be computationally expensive and less interpretable. Furthermore, the majority of these studies do not systematically compare the impact of adding textual features across fundamentally different model types, making it difficult to determine which learning paradigm benefits most from the inclusion of text data. However, this research is different in that it is based on TF-IDF vectorization, a computationally efficient and interpretable feature extraction technique, and its effectiveness is tested using three interpretable and commonly used algorithms: Ridge Regression, Random Forest, and XGBoost. By including a regularized linear model alongside both a bagging-based and a boosting-based ensemble method, this study enables a structured comparison across different learning paradigms, providing insight into which type of model benefits most from the addition of textual listing descriptions.

3 Methodology

3.1 Data Preparation

3.1.1 Data cleaning and filtering

The Oregon real estate dataset [11] before augmentation was a 10,404 by 12 matrix, where each row represented a separate real estate listing that was sold in 2025. The target variable is the price at which each listing was sold. This price was logged to account for the natural skew in house prices. During exploratory data analysis, it was discovered that there were rows with missing features or the target feature. All rows with missing features or missing the target feature were removed. An exception was made for rows where the only missing value was that of garages on the property, as they were assumed to be 0, as these listings typically represented apartment units that did not have garages. Following this, the property type "land" was removed as all properties with the listing as land only consisted of the lot's square footage and the raw text listing description, which was deemed an insufficient amount of data for training the models that did not take raw text into account. One column was also removed, namely "Sub property type" as it was deemed redundant and consisted of missing values for every entry that was not an apartment. After cleaning, the dataset consisted of 8,511 listings.

3.2 TF-IDF

TF-IDF [10] is split into two parts, Term Frequency (TF) is calculated as the frequency of a term within a given document when compared to the total word count. Inverse Document Frequency (IDF) is used to reduce the weightage of terms that occur frequently across documents because they carry less information. In other words, TF-IDF(t,d) = TF(t,d) * log(N/df(t)), where N is the number of documents in the dataset and df(t) is the number of documents that contain the term t. Thus, TF-IDF is calculated as the product of both values. This results in a sparse matrix where every row corresponds to a real estate listing and every column corresponds to a term in the dictionary. This is directly compatible with ML algorithms because it treats text input as a structured numerical input, similar to square footage, number of bedrooms, and type. In terms of house price prediction, TF-IDF can potentially pick up on unique listing terms such as

"renovated kitchen" or "waterfront view" which may have significant predictive power that is not possible with structured features alone. However, it is well known that the major drawback of the TF-IDF is that it does not take into account the order of the terms and the relationships between the terms.

3.3 Feature Engineering

The remaining features consisted of numerical, categorical, and natural language columns. Numerical columns were normalized using the MinMaxScaler implemented by scikit-learn using the default settings, the scalar was only applied over the training data to prevent data leakage. The two remaining categorical columns were encoded using one-hot encoding, turning each category into a binary column. The raw text was vectorized using the term Frequency-Inverse Document Frequency (TF-IDF), which converts the raw text into a numerical sparse matrix.This was only fit on the actual training data with the testing and validation data using the now calculated values. After cleaning and preprocessing, the final data set consisted of 8,511 samples in 15 columns. The increase from 12 to 15 columns originates from the one-hot encoding, as there were four categories offset by the removal of the subproperty column. For every model the dataset was split into 3 sections in a 70%/15%/15% training, validation, test split with the seed 42 using the sklearn split method.

3.4 Models

Each model was trained twice, one with the text vectors from TF-IDF and the other without the text data as a whole; this was done to see the effects of TF-IDF on different models compared to how they performed without. The three models that were selected for this study, Ridge Regression, Random Forest and XGBoost. Ridge Regression was chosen as a regularized linear model, where its L2 penalty makes it particularly suited for high dimensional, potentially correlated features such as those produced by TF-IDF. This allows for a direct comparison between a regression based approach and tree based ensemble methods. Both Random Forest and XGBoost were selected as tree based ensemble models. However, they are based on two completely different ensemble learning approaches. One is based on bagging, where multiple decision trees are learned individually on random subsets of the data and their predictions are combined by averaging. On the other hand, XGBoost is based on boosting, where multiple decision trees are learned sequentially such that each tree attempts to correct the mistakes made by the previous tree. By using both models in this study, it is possible to compare whether bagging is better than boosting in incorporating TF-IDF vectors. Finally, using three models facilitates a structured evaluation between linear regression and tree based ensemble learning algorithms and their two learning approaches.

3.4.1 XGBoost

According to [5], XGBoost, or extreme gradient boost, is a tree boosting algorithm specialising in scalability and accuracy in large datasets compared to other gradient tree boosting algorithms. It is a tree ensemble model that uses n additive functions, with each function representing an independent regression tree. Learning the functions happens by minimizing an objective that combines a loss function with a penalty term that helps the model avoid overfitting. Part of its optimisation is that it uses an exact greedy algorithm during training to add new trees into the ensemble that improve the objective the most, rather than trying to optimise already existing trees.

For this study, the model used was XGBRegressor, a regression model in the XGBoost library. The regressor was trained using the “gbtree” booster, as this booster allowed training with a GPU. Hyperparameter tuning was done using Optuna, a common optimizer. In addition to the built-in hyperparameters, the optimiser was also given access to the maximum feature cutoff of the TF-IDF vectoriser. The regressor was set to stop if there was no improvement within 50 rounds or the n_estimator hyperparameter, which spanned between 1 and 1000 rounds, whichever came first. Every Optuna study was saved, with the best model being retrained afterwards to allow recovery. Each study ran for 1500 trials or until the improvements plateaued to allow the optimizer to find the optimum based off of the MSE.

3.4.2 Random Forest Regressor

Like XGBoost, the Random Forest Regressor uses a forest of tree predictors to do regression tasks.According to [6], it differs in the way it makes these predictions. Instead of constructing each tree sequentially like a boosting model, Random forest works by creating a collection of independent tree predictors, each one using a random subset of the data and a random subset of features. Each predic-

tor then predicts its own value based on the data. To get the algorithm's prediction, the unweighted average of the values is taken. In particular, the Random Forest Regressor leverages the Law of Large Numbers: as the number of trees in this collection increases, the mean predictions converge towards the model's true prediction as the variance of the ensemble decreases.

The Random Forest Regressor used in this study comes from the Sklearn library. Unlike the other models, grid search was used to tune the hyperparameters. The max feature cutoff was left unbounded as during testing for XGBoost and RidgeCV it was discovered that all models would trend to the highest possible max features.Grid search was applied across all remaining hyperparameters in scikit-learn's RandomForestRegressor implementation. Grid search tested 20 models on the validation data and the model with the lowest RMSE was selected.(uses 6) The Random Forest Regressor used in this study comes from the Scilearn library. Unlike the other models, grid search was used to tune the hyperparameters. The max feature cutoff was set to infinite as during testing for XGBoost and RidgeCV it was discovered that all models would trend to the highest possible max features. Grid search was given access to all other hyperparameters in the Scilearn Random Forest Regressor class. Grid search tested 20 models on the validation data and the model with the lowest RMSE was selected.

3.4.3 Ridge Regression

Ridge regression is a linear model based on the standard Ordinary Least Squares (OLS) model, which attempts to minimize $\|y - X\beta\|^2$ [7]. Ridge regression differentiates itself from OLS in that it adds an L2 penalty function to the loss function to discourage overfitting. The loss function of Ridge regression is then $\|y - X\beta\|^2 + \lambda\|\beta\|^2$, with $+ \lambda\|\beta\|^2$ being the penalty function [8]. This penalty function penalises the size of the coefficients β. Using this loss function enables us to reduce the variance of the model, albeit at the expense of increased bias, which in turn allows the model to generalize more effectively across the data. λ is then the hyperparameter which has control over how much the coefficients get penalized. After taking the derivative of the loss function setting it to zero, and solving for β the estimator $\beta = (X'X + \lambda I)^{-1}X'y$ emerges which determines the optimal coefficients for each feature[7]. Ridge regression was trained using Ridge from Sklearn. Optuna was once again used to tune the hyperparameters λ and the max feature cutoff for the TF-IDF vectorizer as it uses a more intelligent Bayesian optimization strategy. Each Optuna study was given 50 trials and the model with the lowest RMSE on the test set was selected.

3.4.4 Baseline

For the baseline this study uses the mean as a predictive model. For this the mean is taken over the target data on the training data. This mean is then used as a prediction for the test data. Baseline models like this are very useful to test the effectiveness of other models and to demonstrate that models are learning actual data and not just the mean.

3.5 Evaluation metrics

Every model was evaluated on R^2 , RMSE and MAE on the test set. This produced 7 sets of data with 2 per each model and one for the mean as the mean does not differ between the use of text or not.

3.5.1 R^2

R squared measures the amount of variance explained by the model using: $1 - (\Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2)$ with y the actual value, $\hat{y}$ the prediction from the model and $\bar{y}$ the mean of all actual values[9]. The value between 1 and 0 with 1 meaning your model can explain every bit of variation and 0 meaning that the model does not perform better than the mean in explaining variation. It was chosen as it is important to show how much of the data the model is accounting for during testing.

3.5.2 RMSE

Root Mean Square Error is a modification of the normal MSE and uses the equation: $(1/n\ \Sigma(y - \hat{y})^2)$ with yi being the real value that the model is trying to predict in the ith position, $\hat{y}$ the prediction made by the model[9]. It shows how large models given errors are with large errors being weighted more than smaller errors. Normally it would also provide directly interpretable numbers on the same scale as the target value however due to the price being logged previously so will the values of the RMSE. Unlike R^2 RMSE does not have a limit on values in the positive and must thus be interpreted based on the data provided to the model. It was chosen as it's a common way to measure the size of errors a model makes and thus deemed necessary to include.

3.5.3 MAE

Mean Absolute Error with the equation $1/n \Sigma | y - \hat{y} |$ which returns the mean of the absolute error [9].It again has no upper limit to the values it can give with lower values meaning better fitting models. It's in the same scale as the pricing data which allows for it to be easily interpretable as well as being a simple metric to add.

4 Results

After training the models and evaluating them on the test set this data was collected. With the R^2 and RMSE being the primary metrics.

The results for the baseline show a R^2 of -0.0008, this indicated that the baseline model was unable to account for any of the variation indicating the need for machine learning algorithms.

The results show that for every case, the models tested with text performed better at the regression task compared to the models, with noticeable changes in both the RMSE as well as R^2.

Out of these models XGBoost with TF-IDF had the best overall performance with a R^2 of 0.7021 and a RMSE of 0.3531. XGBoost also had the greatest improvement in R^2 when TF-IDF was used with a change of +0.2868 compared to Ridge with +0.2161 and Random Forest with +0.0577. In terms of RMSE reduction Ridge showed the largest improvement with -0.0921, followed by XGBoost with -0.0622 and RF with -0.0291.

It is worth noting that Random Forest achieved the highest R^2 performance without text as well as had the lowest improvement in R^2. When evaluating MAE it was discovered that the MAE of Ridge without text features achieved the lowest MAE out of all models despite text features improving performance across all other metrics as well as models

5 Discussion

The results show indication that including the description of the property in regression models is more effective in accurately predicting the value of that property than in creating regression models that do not take into account text. However additional steps will need to be taken to prove statistical significance in the difference between the inclusion of the description, and to decisively prove which model is best suited to this task. For example, is the difference between a R^2 of 0.7021 and 0.6668 enough to select the XGBoost model over the Ridge Regression when there are drawbacks to XGBoost that are not taken into account by the numbers alone. XGBoost is a method where multiple decision trees learn sequentially, where each tree maps an input to a fixed leaf weight [5]. Since splits are learned from the training data, any input outside of the range of the training data will fall to the right-most partition and therefore will receive a constant prediction. This means XGBoost cannot extrapolate beyond the range of values it is trained on, unlike Ridge Regression which is a linear classifier and therefore able to extrapolate.

When considering the Random Forest model, the model without the text description achieved the highest R2 performance compared to the other models without the text description. However it had the lowest improvement when the models were tested against text descriptions. This meant it performed the worst compared to the other two models, however worked better than the other models when there was no text description for the dataset. This could be explained by the fact that Random Forest does not change to unvaried transformations of the input features, meaning that for TF-IDF, where the term frequency is multiplied by the ratio between total number of documents and the number of documents containing the term, the Random Forest model is unaffected [12]. Therefore the Random Forest model is more dependent on the numerical features of the data, and did not have much change when the text description was added to the features.

The mean absolute error (MAE) functions similarly to the mean squared error (MSE) where the closer the value to zero the better performing the model is. As established, the overall performance of all three models improved with the addition of TF-IDF, however for the ridge regression model, the MAE increased as seen in table 4. The MAE is the sum of the absolute value of the difference between the actual value and the predicted value, since it does not square the value of the difference before summing it like in MSE, MAE is not as sensitive to larger errors. Since Ridge Regression is a linear model, it makes more small errors and few large errors, unlike the decision tree based Random Forest and XGBoost models. This affects the MAE score more for Ridge Regression, which could be a reason Ridge Regression stands out when compared to the other models in this evaluation metric.

A limitation of this implementation of text descriptions in these models is that TF-IDF treats all of the words independently of each other, and loses the semantic meaning of the words. In a closed

	Baseline	Ridge No Text	Ridge Text	RF No Text	RF Text	XGBoost No Text	XGBoost Text
R^2	-0.0008	0.4507	**0.6668**	0.5574	**0.6151**	0.4153	**0.7021**
RMSE	0.6472	0.4663	0.3734	0.4304	0.4013	0.4153	0.3531
MSE	0.4188	0.3050	0.1394	0.1850	0.1613	0.1725	0.1247
MAE	0.4660	0.2175	0.2368	0.2822	0.2474	0.2758	0.2257

Table 1: Evaluation metrics for fine-tuned models and baseline.

	RMSE Without TF-IDF	RMSE With TF-IDF	Change
Ridge	0.4663	0.3734	-0.0929
Random Forest	0.4304	0.4013	-0.0291
XGBoost	0.4153	0.3531	-0.0622

Table 2: RMSE change from adding TF-IDF matrices by model

world this may not make a lot of difference in performance, however in the real world case of real estate the semantic meaning of words is directly relevant to the price of the property. For example, a phrase like "Newly renovated" would be split apart and meanings derived from the two separate words "newly" and "renovated", which could affect how the model predicts the value of that property. A way to overcome this limitation would be to use a different text embedding technique like BERT or Word2Vec.

6 Conclusion

This paper aimed to answer the question: "Does adding written house descriptions improve the predictive accuracy of ML models for house price prediction" with a sub question "Which model out of ridge regression,random forest, XGBoost shows the greatest performance improvement when listing descriptions are added to structured features?"

For the main question it is apparent that across Ridge regression, Random Forest Regressor and XGBoost did improve with an average of -0.0614 improvement in Root Mean Square Error, an average change of 0.18 in R^2. When it came to MAE there was an anomaly where the MAE of Ridge Regression went up with 0.0193 compared to without text. The RFR and XGBoost followed the previous trend with an average of -0.0425. This shows there is an improvement when providing TF-IDF encoded descriptions to each model.

For the subquestion, XGBoost with text shows the best overall performance with a R^2 of 0.7021, a RMSE of 0.3531 and a MAE of 0.2257. The addition of TF-IDF on XGBoost also influenced the model the most with the highest changes in both MAE with -0.0501 and R^2 with 0.2868 as well as the second largest change in RMSE with -0.0622. This shows that adding this extra text data to these models improves their performance.

Future research should investigate whether other vectorizers have the same effects as well as possible transformer based models such as BERT. As mentioned in the discussion the use of other text models could improve the performance of the models as TF-IDF has the drawback of treating every word as independent of each other.

Other datasets with more instances and wider ranges should also be tested to verify generalisability. The anomaly in MAE for ridge regression should be investigated as well as a deeper look into if Ridge Regression actually improves off of text data.

7 Bibliography

[1] K. Baur, M. Rosenfelder, and B. Lutz, "Automated real estate valuation with machine learning models using property descriptions," Expert Systems with Applications, p. 119147, Oct. 2022, doi: https://doi.org/10.1016/j.eswa.2022.119147.

[2] H. Zhang, Y. Li, and P. Branco, "Describe the house and I will tell you the price: House price prediction with textual description data," Natural Language Engineering, vol. 30, no. 4, pp. 1–35, Jul. 2023, doi: https://doi.org/10.1017/s1351324923000360.

[3] Dr. Y. Shen and Dr. Y. Han, "A Natural Language Processing Model for House Price Forecasting - CURF," CURF, Mar. 18, 2025. https://curf.clemson.edu/technology/a-natural-language-processing-model-for-house-price-forecasting/

[4] H. Hasan , A. Jahan, and M. E. Ali, "A Multi-Modal Deep Learning Based Approach

	R^2 without TF-IDF	R^2 with TF-IDF	Change
Ridge	0.4507	0.6668	0.2161
Random Forest	0.5574	0.6151	0.0577
XGBoost	0.4153	0.7021	0.2868

Table 3: R^2 change from adding TF-IDF matrices by model

	MAE without TF-IDF	MAE with TF-IDF	Change
Ridge	0.2175	0.2368	0.0193
Random Forest	0.2822	0.2474	-0.0348
XGBoost	0.2758	0.2257	-0.0501

Table 4: MAE change from adding TF-IDF matrices by model

for House Price Prediction," Arxiv.org, 2024. https://arxiv.org/html/2409.05335v1

[5] T. Chen and C. Guestrin, "XGBoost: a Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16, vol. 1, no. 1, pp. 785–794, Aug. 2016, doi: https://doi.org/10.1145/2939672.2939785.

[6] M. R. Segal, "Machine Learning Benchmarks and Random Forest Regression," Escholarship.org, 2004. https://escholarship.org/uc/item/35x3v9t4

[7] A. E. Hoerl and R. W. Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems," Technometrics, vol. 42, no. 1, p. 80, Feb. 2000, doi: https://doi.org/10.2307/1271436.

[8] W. Van Wieringen, "Lecture notes on ridge regression," 2021. Available: https://arxiv.org/pdf/1509.09169

[9] D. Chicco, M. J. Warrens, and G. Jurman, "The Coefficient of Determination R-squared Is More Informative than SMAPE, MAE, MAPE, MSE and RMSE in Regression Analysis Evaluation," PeerJ Computer Science, vol. 7, no. 5, p. e623, 2021, doi: https://doi.org/10.7717/peerj-cs.623.

[10] K. SPARCK JONES, "A STATISTICAL INTERPRETATION OF TERM SPECIFICITY AND ITS APPLICATION IN RETRIEVAL," Journal of Documentation, vol. 28, no. 1, pp. 11–21, Jan. 1972, doi: https://doi.org/10.1108/eb026526.

[11] Kanchana1990, "Real Estate Data Oregon 2026," Kaggle.com, 2026. https://www.kaggle.com/datasets/kanchana1990/real-estate-data-oregon-2026 (accessed Mar. 27, 2026).

[12] R. Perry et al., "Random forests for adaptive nearest neighbor Estimation of Information-Theoretic quantities," arXiv (Cornell University), Jun. 2019, doi: 10.48550/arxiv.1907.00325.

MLVU information sheet

Please include this page in your report either at the start or at the end, before the appendix. Do not change the formatting.

Group number
43

Authors

name	student number
Alexander Wood	2860328
Natasha Kanth	2835811
Pradakshina Sharma	2839835

Software used .

VS Studio Code. IDE was used
Xgboost, library containing the XGBRegressor used in the study
Pandas, used to import the CSV and create dataframes
Sklearn, used for preprocessing, splitting the data, as well as Ridge Regression and Random Forest models.
Joblib, used to store the scalers.
Scipy, used for its sparse matrices to save space with the TF-IDF vectorization as well as H-stacks
Optuna, used to optimize hyper parameters
Matplotlib, used to create graphs
Numpy, used for mathematics such as creating the mean model as well as creating arrays
Google docs, used to create a proof version of the report before moving to overleaf
Overleaf. Used to make the report
Google Sheets used to create graphs and tables

Use of AI tools Claude: Claude was used for bug fixing, making code more readable, as well as general feedback on the report.
Grammarly: Grammarly provides tools to check grammar in documents and suggest fixes, parts of the document were provided to grammarly to correct on spelling, punctuation and the flow of sentences.
Google docs now uses AI to do spellcheck and grammar checking within documents which was used during the creation of the report.
Overleaf, Overleaf like google docs uses an llm called writeful to create text suggestions to improve clarity which was used.

Link to code (optional) If you would like to share your code, you can link to your repository here.

Group disagreements (optional) If there are any disagreements in the group, you may **not** remove a student from the author list without their consent. You should bring disagreements to our attention early. As a last resort, you can describe any grievances here.

A Appendix

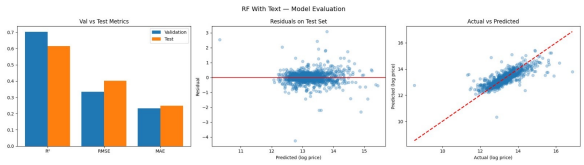

Figure 1: Random Forest with Text

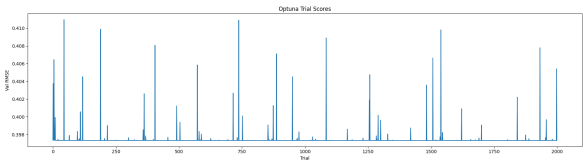

Figure 2: Optuna trial scores Ridge Regression without Text

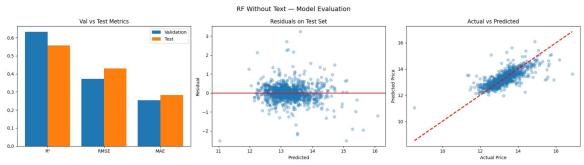

Figure 3: Random Forest without Text

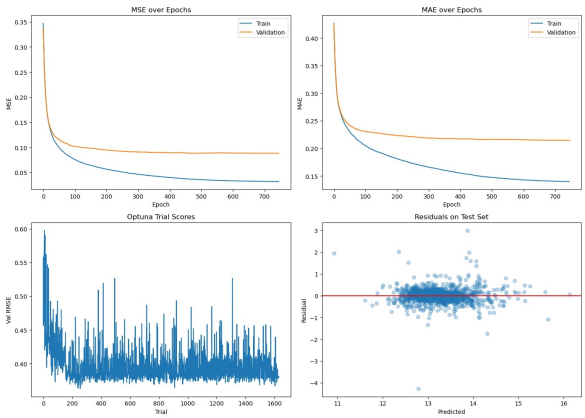

Figure 4: XGBoosting without text

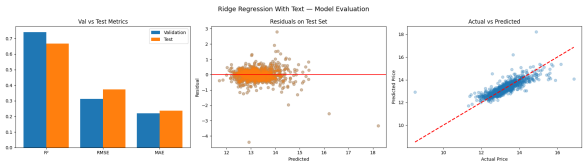

Figure 5: Ridge Regression with Text

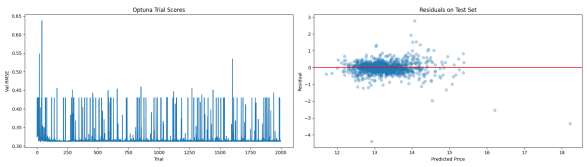

Figure 6: Optuna Trial Scores and Residuals Ridge Regression with Text

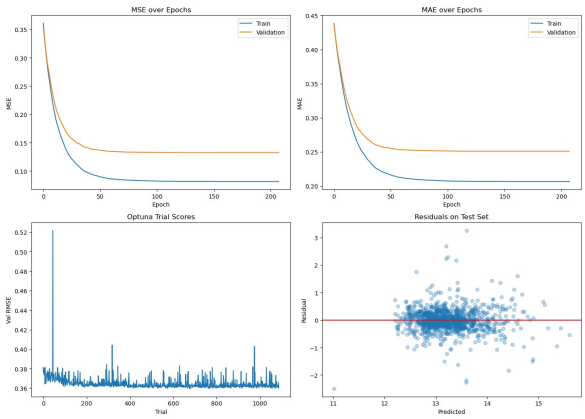

Figure 7: XGBoosting with Text

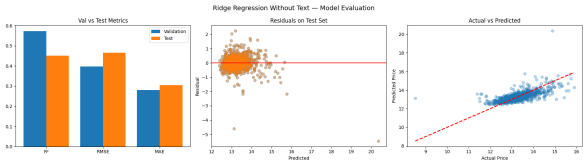

Figure 8: Ridge Regression without text

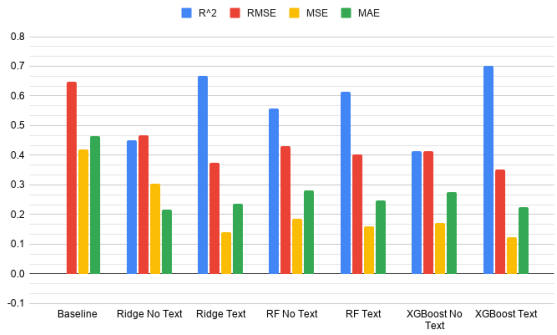

Figure 9: Evaluation metrics against all models with and without text